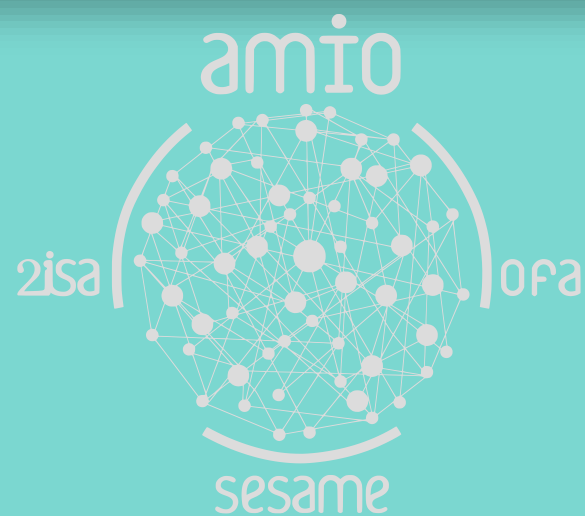


2026

Dossier de Projet : PAE

Concepteur Développeur d'Applications



Ceyhane YILMAZ

2iSA – CDA09

19/02/2026

PRESENTATION PERSONNELLE	3
PRESENTATION DE LA FORMATION CDA	4
GENERALITES DU PROJET PRESENTE.....	5
Liste des compétences du référentiel couvertes par le projet.....	6
EXPRESSION DES BESOINS DU PROJET.....	7
Contexte dans lequel s'inscrit ce projet	7
Présentation de l'équipe projet	8
Besoins fonctionnels identifiés	8
GESTION DU PROJET.....	9
Organisation et suivi de l'activité	9
Méthodologie adoptée	9
État d'avancement	9
ANALYSE DU BESOIN ET SPECIFICATIONS FONCTIONNELLES	11
Acteurs identifiés.....	11
Cas d'utilisation principaux.....	11
SPECIFICATIONS TECHNIQUES	12
Choix de Polars plutôt que Pandas	12
PRESENTATION DETAILLEE DES COMPOSANTS DEVELOPPES	13
Vue d'ensemble de l'architecture	13
Diagramme de séquence – Ingestion d'une table	14
PARCOURS TECHNIQUE – RECIT CHRONOLOGIQUE	15
Phase 1 – Pipeline ETL pg_azure_pipeline : mon premier chantier.....	15
Phase 2 – Meltano et Oracle Data API : deux pistes en parallèle	19
Phase 3 – Découverte tardive de DLT (DLT-HUB)	24
Phase 4 – Intégration dans sync_oracle_postgresql_clean : le projet de Marine	26
Phase 5 – Stratégies d'ingestion testées	27
SECURITE DE L'APPLICATION	29
Protection contre l'injection SQL.....	29
Gestion des secrets	29
Rate limiting.....	29
Compression des réponses.....	29
Validation des entrées	30
PLAN DE TEST	31
Stratégie de test	31
Exemple de tests unitaires	31
Couverture de tests	32
ARCHITECTURE DE DEPLOIEMENT CIBLE.....	33
Procédure de déploiement de l'API Oracle	33
Pipeline Azure DevOps (CI/CD)	33
CONFORMITE ET RESPONSABILITE NUMERIQUE	35
1. Respect du Règlement Général sur la Protection des Données (RGPD).....	35
2. Principes d'éco-conception (Green IT)	35
DIFFICULTES RENCONTREES ET EXPLORATION	37
Difficultés liées à Meltano et Singer	37

DECOUVERTE TARDIVE DE DLT	37
DIFFICULTES ORGANISATIONNELLES.....	38
VEILLE TECHNOLOGIQUE – STACK DATA MODERNE	40
STACK RECOMMANDEE	40
BILAN	42
APPORTS ET APPRENTISSAGES	42
DIFFICULTES SURMONTÉES	42
POINTS D’AMELIORATION IDENTIFIES	43
PERSPECTIVES.....	43
REMERCIEMENTS	44
ANNEXES	45
ANNEXE A – STRUCTURE COMPLETE DU PROJET ORACLE_API.....	45
ANNEXE B – STRUCTURE COMPLETE DU PROJET PG_AZURE_PIPELINE	45
ANNEXE C – STRUCTURE DU PROJET SYNC_ORACLE_POSTGRESQL_CLEAN.....	45
ANNEXE D – ENDPOINTS DE L’ORACLE DATA API.....	46
ANNEXE E – GLOSSAIRE.....	46
ANNEXE F – DIAGRAMME DE SEQUENCE DU PIPELINE D’INGESTION	48

Présentation personnelle

Je m'appelle **Ceyhane YILMAZ** et je me forme actuellement pour devenir **Concepteur Développeur d'Applications Full-Stack**. Depuis près de dix ans, je nourris un réel intérêt pour la résolution de problèmes informatiques : automatiser des tâches, traquer des bugs, explorer de nouvelles technologies... Chaque défi m'a permis d'affiner mon intuition et d'acquérir une solide logique de développeur.

Après une première année de licence en informatique à Perpignan, j'ai choisi de réorienter mon parcours vers une approche plus concrète du développement. Cette période, marquée par la pandémie, m'a aussi permis de travailler sur des projets personnels : contribution à un jeu vidéo (*Soundsphere*) et création de scripts d'automatisation. Ces expériences m'ont appris à travailler en autonomie, à lire du code existant, à contribuer à des projets open-source et à comprendre les enjeux de la collaboration dans un environnement distribué.

Aujourd'hui, au sein de **2iSA**, je mets à profit mes expériences pour consolider mes compétences techniques et concrétiser mes ambitions professionnelles dans le développement logiciel. Je suis motivé à transformer ma curiosité en une véritable expertise. Ce stage en milieu hospitalier représente pour moi une opportunité unique de confronter mes compétences de développeur à un domaine métier exigeant, où la fiabilité des données a un impact direct sur la qualité de soin.

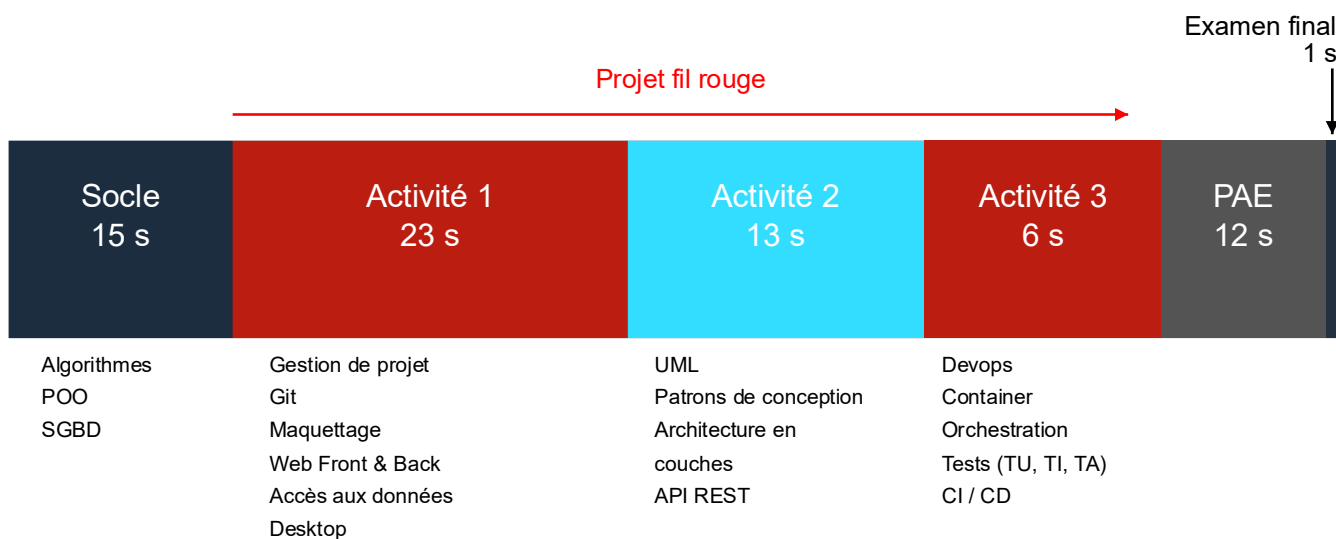
Mon parcours atypique – entre autodidaxie, licence universitaire et formation professionnelle – m'a doté d'une adaptabilité que j'ai pu mettre à profit durant ce stage. J'ai dû apprendre un domaine entièrement nouveau (le *data engineering*), comprendre un système d'information hospitalier complexe, et livrer des composants logiciels fonctionnels dans un délai contraint. C'est cette polyvalence, acquise au fil des années, qui m'a permis de relever ce défi.

Présentation de la formation CDA

La formation est structurée en **3 activités** de sorte à permettre l'acquisition des compétences professionnelles du titre Concepteur Développeur d'Applications (CDA) selon une progression pédagogique adaptée au public accueilli.

En parallèle des apprentissages, dans le but de **renforcer la pratique professionnelle**, les stagiaires mènent un projet fil rouge, une mise en situation professionnelle qui les amène à construire une application en plusieurs étapes.

Le parcours se déroule sur 69 semaines dont 12 semaines de Période d'Application en Entreprise (PAE) et 2 semaines passerelles.



Activité 1 – Développer une application

- Gérer son environnement de développement
- Développer une interface utilisateur
- Développer les composants une application
- Collaborer à la gestion d'un projet informatique

Activité 2 - Concevoir et développer une application organisée en couches

- Analyser les besoins et maquetter l'application
- Définir l'architecture logicielle d'une application
- Concevoir et mettre en place une base de données relationnelle
- Développer des composants d'accès aux données SQL et NoSQL

Activité 3 – Préparer le déploiement d'une application

- Préparer et documenter le déploiement d'une application
- Préparer et exécuter les plans de tests d'une application
- Contribuer à la mise en production dans une démarche DevOps

En transverse :

- Utiliser l'anglais dans son activité professionnelle
- Actualiser et partager ses compétences en développement informatique
- Communiquer dans le contexte professionnel

Généralités du projet présenté

Le **Projet de stage** constitue le pilier final de la formation CDA. Il simule une situation réelle de développement dans laquelle je dois concevoir des applications et les encapsuler.

Objectifs pédagogiques :

- Il vise à **apporter de la cohérence aux apprentissages** en mobilisant l'ensemble des compétences acquises au fil des modules.
- Il sert de support principal pour **l'examen du titre professionnel**, permettant aux étudiants de démontrer l'acquisition des compétences requises.
- Il contribue à l'alimentation du **Dossier Professionnel (DP)**.

Structure :

Le projet est divisé en deux parties principales, chacune avec des livrables spécifiques à présenter lors de l'évaluation :

- **Partie 1** : se documenter sur l'environnement de travail, lire le cahier des charges et l'analyse des besoins ou les réaliser, produire des diagrammes UML.
- **Partie 2** : se familiariser avec les outils d'ingénierie des données, développer un pipeline d'ingestion, deux API REST, dont une pour contrôler le pipeline et enfin, faire des plans de test.

Liste des compétences du référentiel couvertes par le projet

Ce stage me permet de mettre en œuvre et de faire évaluer les compétences suivantes du **Référentiel Emploi Activités Compétences (REAC)** :

- **Développer une application sécurisée**
 - Installer et configurer son environnement de travail en fonction du projet
 - Développer des composants métier
 - Contribuer à la gestion d'un projet informatique
- **Concevoir et développer une application sécurisée organisée en couches**
 - Analyser les besoins et maquetter une application
 - Définir l'architecture logicielle d'une application
 - Concevoir et mettre en place une base de données relationnelle
 - Développer des composants d'accès aux données SQL et NoSQL
- **Préparer le déploiement d'une application sécurisée**
 - Préparer et exécuter les plans de test d'une application
 - Préparer et documenter le déploiement d'une application
 - Contribuer à la mise en production dans une démarche DevOps
- **Compétences transverses**
 - Communiquer en français et en anglais
 - Mettre en œuvre une démarche de résolution de problème
 - Apprendre en continu

Ce projet m'a permis de couvrir ces compétences de manière transversale. L'architecture en couches de mon pipeline ETL (Extract – Transform – Load), la sécurisation des accès aux données via Azure Key Vault, la validation des identifiants SQL contre l'injection, les stratégies de chargement interchangeables (pattern Strategy) et le travail de documentation constituent autant de preuves tangibles de la maîtrise de ces compétences.

Expression des besoins du projet

Contexte dans lequel s'inscrit ce projet

Ce projet s'inscrit dans le cadre de ma formation de CDA et constitue l'application des compétences en entreprise. Son but premier est pédagogique, mais il répond aussi à un besoin réel et concret du Centre Hospitalier de Mende.

Le service des urgences utilise le PGI **HM** (*Hopital Manager*), édité par **Softway**, un fournisseur spécialisé dans les systèmes d'information hospitaliers. Ce logiciel s'appuie sur une base de données **Oracle** qui contient l'ensemble des données opérationnelles du service : passages aux urgences (RPU), diagnostics, mouvements de patients, séjours, absences du personnel, etc.

L'objectif métier est de **mettre à disposition ces données dans une base PostgreSQL hébergée sur Azure**, afin de permettre leur exploitation analytique par l'équipe de data management. La base Oracle de production ne peut pas être sollicitée directement pour des requêtes analytiques lourdes : cela impacterait les performances du logiciel métier utilisé en temps réel par les soignants. Il faut donc **répliquer les données** vers un entrepôt dédié.

Ce besoin s'inscrit dans une démarche plus large de **data management hospitalier**, portée par Marine Crognier et l'équipe du pôle régulation et intelligence artificielle. L'expression des besoins formalisée dans le document « Expressions des besoins Data management URGENCES » décrit précisément le périmètre des tables à synchroniser (environ 90 tables issues de la base HM), les contraintes de volumétrie et les exigences de fraîcheur des données.

Mise en Situation Professionnelle : ce stage m'a placé dans une position réelle de développeur junior intégré à une équipe projet. J'ai dû comprendre le besoin métier, analyser la documentation existante, choisir les technologies appropriées et développer des composants logiciels fonctionnels.

Application des Compétences : l'objectif est de mettre en œuvre les compétences acquises durant la formation, notamment celles liées à l'architecture en couches, au développement d'API REST et à la préparation du déploiement.

Évaluation Formative : le projet sert de support principal à l'évaluation finale. Mon travail sera évalué pour démontrer l'acquisition des compétences du REAC.

Constitution du Dossier Professionnel : les réalisations (code, documents de conception, plans de test et de déploiement) sont destinées à alimenter mon Dossier Professionnel (DP).

Présentation de l'équipe projet

Le projet est porté par une équipe restreinte au sein du Centre Hospitalier de Mende :

Marine Crognier – Maître de stage et porteuse du projet

Marine Crognier est la cheffe de projet *Data Management* au sein du pôle régulation et intelligence artificielle du Centre Hospitalier. C'est elle qui a rédigé l'expression des besoins et le cahier des charges fonctionnel qui cadrent ce projet. Son profil combine une expertise en gestion de données hospitalières et une vision stratégique de l'utilisation de la data pour améliorer la prise en charge aux urgences. Elle joue le rôle de **Maîtrise d'Ouvrage** (MOA) : elle définit les besoins fonctionnels, valide les livrables et oriente les choix techniques. Au quotidien, nos échanges se faisaient principalement via Microsoft Teams, avec des points réguliers pour suivre l'avancement et lever les blocages.

Marine avait déjà initié un travail de synchronisation Oracle → PostgreSQL avant mon arrivée, en utilisant Python avec les bibliothèques `cx_Oracle`, `SQLAlchemy` et `Pandas`. Ce travail constituait un socle que j'ai repris, amélioré et étendu dans le cadre de mon stage. Le projet `sync_oracle_postgresql_clean` est directement issu de cette base.

Ceyhane YILMAZ – Stagiaire développeur (moi-même)

Mon rôle était celui d'un développeur en charge de la conception, du développement et du test des composants logiciels nécessaires à la chaîne d'ingestion de données. J'étais responsable de la **Maîtrise d'Œuvre** (MOE) : traduire les besoins fonctionnels en solutions techniques, coder les applications et documenter le travail.

L'équipe élargie

Le projet s'inscrivait dans un contexte plus large impliquant le service informatique de l'hôpital (pour les accès réseau et les autorisations), l'administrateur de la base Oracle HM et l'équipe Azure (pour la gestion de l'infrastructure cloud). Ces interlocuteurs étaient sollicités ponctuellement, notamment pour la configuration d'Azure Key Vault et les accès réseau à la base Oracle.

Besoins fonctionnels identifiés

À partir du cahier des charges fonctionnel et de l'expression des besoins, j'ai identifié les fonctionnalités clés suivantes :

1. **Extraction des données** : récupérer les données depuis la base Oracle HM via une méthode sécurisée et performante.
2. **Transformation** : normaliser les types de données (dates, nombres, BLOBs) pour assurer la compatibilité avec PostgreSQL.
3. **Chargement** : insérer ou mettre à jour les données dans la base PostgreSQL Azure, en respectant l'intégrité référentielle.
4. **Incrémentalité** : ne récupérer que les données modifiées depuis la dernière exécution pour minimiser le volume de transfert.
5. **Résilience** : gérer les erreurs de connexion, les timeouts et les cas limites sans perte de données.
6. **Observabilité** : fournir des métriques et des logs pour surveiller le bon fonctionnement du pipeline.
7. **Extensibilité** : permettre l'ajout de nouvelles tables sans modification du code, via une configuration déclarative (JSON).

Gestion du projet

La gestion de projet est une compétence évaluée dans le REAC (CP4 – Collaborer à la gestion d'un projet informatique). Elle a impliqué :

- Planification et suivi des tâches
- Communication au sein de l'équipe via Microsoft Teams
- Récapitulatifs d'activité quotidiens envoyés à la maîtresse de stage
- Itérations sur l'architecture suite aux retours et aux découvertes techniques

Organisation et suivi de l'activité

La gestion de ce projet de stage a débuté par la lecture de l'analyse des besoins déjà établie par Marine Crognier, suivie d'une phase d'exploration des outils d'ingénierie des données. Conscient de la nécessité d'un suivi structuré, j'ai toutefois opté pour une approche plus exploratoire dans un premier temps, en « plongeant dans une piscine d'information » en guise d'immersion dans le métier du data engineering.

Cette approche, si elle m'a permis de découvrir un large éventail d'outils et de techniques, a aussi engendré des difficultés d'organisation. J'ai passé du temps sur des pistes qui se sont avérées infructueuses (Meltano, tentatives de modification de connecteurs Singer) avant de converger vers une solution stable. Avec le recul, une phase de cadrage technique plus rigoureuse en début de projet m'aurait permis d'être plus efficace.

Les récapitulatifs d'activité quotidiens, envoyés à Marine via Teams, m'ont permis de garder un fil conducteur malgré cette exploration non linéaire. Ils constituaient à la fois un outil de communication et un journal de bord technique.

Méthodologie adoptée

Étant le seul développeur sur ce projet, je n'ai pas mis en place un framework agile formel (Scrum, Kanban). J'ai plutôt adopté une approche itérative informelle :

- **Compréhension du besoin** : lecture des documents fonctionnels, échanges avec Marine
- **Exploration technique** : prototypage rapide de différentes solutions
- **Développement** : implémentation de la solution retenue avec refactoring progressif
- **Validation** : tests unitaires, tests d'intégration, revue avec Marine
- **Documentation** : README, changelog, commentaires dans le code

Le suivi était assuré par :

- Des **points Teams** réguliers avec Marine Crognier
- Des **récapitulatifs d'activité quotidiens**
- Un **gestionnaire de versions** (Azure Repos / Git) pour tracer l'historique du code

État d'avancement

Au moment de cet examen, les livrables suivants sont réalisés :

- **Oracle Data API** : une API REST fonctionnelle construite avec FastAPI, exposant les données de la base Oracle HM via des endpoints REST avec pagination, filtrage par date et multiples formats de réponse (JSON, MessagePack).

- **Pipeline d'ingestion pg_azure_pipeline** : un pipeline ETL en Python orchestrant l'extraction (via l'API REST), la transformation (normalisation des types) et le chargement (via COPY et UPSERT) des données vers PostgreSQL Azure.
- **API de synchronisation sync_oracle_postgresql_clean** : une API REST encapsulant le pipeline de synchronisation, permettant le déclenchement à la demande ou planifié.
- **Pipeline Meltano** : un pipeline déclaratif (YAML) utilisant l'écosystème Singer (tap-rest-api-msdk + target-postgres) – fonctionnel mais abandonné.
- **Pipeline dlt** : un pipeline alternatif utilisant la bibliothèque Python dlt – fonctionnel, testé tardivement.
- **Tests unitaires et d'intégration** pour l'API Oracle et le pipeline.
- **Documentation** : README, documentation d'API, diagrammes d'architecture, changelog.

Analyse du besoin et spécifications fonctionnelles

L'analyse du cahier des charges a permis de définir les acteurs et les fonctionnalités attendues pour ce projet.

Acteurs identifiés

Acteur	Rôle	Interaction avec le système
Data Manager (Marine Crognier)	Définit les tables à synchroniser, valide la qualité des données	Configure les fichiers JSON, consulte les métriques
Pipeline automatisé	Exécute la synchronisation périodiquement	Appelle l'API Oracle, charge les données dans PostgreSQL
Administrateur système	Gère l'infrastructure Azure et les accès réseau	Configure Azure Key Vault, autorise les connexions
Analyste de données	Exploite les données synchronisées pour produire des indicateurs	Requête la base PostgreSQL directement

Cas d'utilisation principaux

1. **Synchronisation incrémentale d'une table** : le pipeline récupère uniquement les lignes modifiées depuis la dernière exécution, les transforme et les charge dans PostgreSQL via un UPSERT.
2. **Synchronisation initiale et mensuelle (full load)** : lors de la première exécution et chaque mois, toutes les données d'une table sont extraites et chargées.
3. **Consultation de l'état de santé** : un endpoint /health permet de vérifier la connectivité de l'API avec la base Oracle.
4. **Consultation des métriques** : un endpoint /metrics expose des métriques Prometheus pour le monitoring.

Déclenchement manuel : l'API de synchronisation permet de déclencher une synchronisation à la demande via un appel HTTP POST.

Spécifications techniques

Pour répondre aux exigences techniques du cahier des charges, j'ai mis en place un environnement de développement cohérent. L'architecture repose sur des technologies modernes et robustes issues de l'écosystème Python.

Élément	Technologie / Outil
Langage principal	python
Framework Back-End	FastAPI
Base de données source	Oracle Database (HM)
Base de données cible	PostgreSQL (Azure)
ORM / Drivers	SQLAlchemy, oracledb, psycopg
Bibliothèque de données	Polars
Gestion des secrets	Azure Key Vault
Sérialisation	JSON, MessagePack
Monitoring	Prometheus (prometheus-fastapi-instrumentator)
Rate limiting	SlowAPI
Logging	Loguru
Validation	Pydantic, JSON Schema
IDE	Visual Studio Code
Contrôle de version	Git / Azure Repos
Modélisation	UML (diagrammes de séquence, d'architecture)
CI/CD	Azure DevOps Pipelines

Choix de Polars plutôt que Pandas

Un choix technique notable a été l'utilisation de **Polars** plutôt que Pandas pour le traitement des DataFrames. Polars est une bibliothèque de manipulation de données écrite en Rust, offrant des performances significativement supérieures à Pandas grâce à :

- L'exécution parallèle native (multi-thread)
- L'évaluation paresseuse (*lazy evaluation*)
- Une empreinte mémoire plus faible (format Apache Arrow)
- Un typage strict qui prévient les erreurs de conversion

Ce choix s'est avéré pertinent dans notre contexte : certaines tables comptent plus de 5 millions de lignes, et la différence de performance entre Polars et Pandas est notable à cette échelle.

Présentation détaillée des composants développés

Vue d'ensemble de l'architecture



Flux de données simplifié :

1. L'Oracle Data API se connecte à la base Oracle HM via un pool de connexions `oracledb` et expose les données sous forme de REST API.
2. Le Pipeline ETL (`pg_azure_pipeline`) consomme cette API pour extraire les données, les transforme (normalisation des types, cast des colonnes) et les charge dans PostgreSQL Azure.
3. La Sync API (`sync_oracle_postgresql_clean`) encapsule le pipeline et expose des endpoints pour déclencher, surveiller et gérer les synchronisations.

Azure Key Vault fournit les secrets (identifiants de bases de données) à tous les composants, évitant le stockage en dur des mots de passe.

Diagramme de séquence – Ingestion d’une table

Le diagramme suivant illustre le flux complet d’ingestion des données pour une table, depuis le déclenchement jusqu’à la confirmation du chargement.

[Voire annexe F](#)

Ce diagramme met en évidence plusieurs aspects importants de l’architecture :

- **La pagination par curseur** (*keyset pagination*) : plutôt que d’utiliser OFFSET/LIMIT (qui devient lent pour les grandes tables), j’utilise un curseur basé sur la colonne de date et la clé primaire. Cela garantit des performances constantes quelle que soit la profondeur de la pagination.
- **Le pattern ETL classique** (Extract → Transform → Load) implémenté avec une séparation claire des responsabilités.
- **La vérification d’intégrité post-chargement** : après chaque chargement, le pipeline vérifie que les données dans PostgreSQL correspondent bien aux données source.
- **La mise à jour du curseur** : le timestamp de la dernière extraction réussie est persisté dans un fichier JSON, permettant la prochaine exécution incrémentale.

Parcours technique – Récit chronologique

Cette section retrace chronologiquement les différentes approches techniques explorées durant le stage. Ce récit est important car il illustre la réalité du développement logiciel : les choix ne sont pas toujours linéaires, et l'exploration de pistes infructueuses fait partie intégrante du processus d'ingénierie.

Mon parcours n'a pas suivi un chemin rectiligne. J'ai d'abord construit mon propre pipeline ETL (`pg_azure_pipeline`), puis exploré Meltano en parallèle de la création de l'API Oracle – cette dernière étant née précisément de mon incapacité à faire fonctionner le connecteur Oracle de Meltano. C'est seulement après que Marine Crognier a comparé mes projets au sien qu'elle a conclu que je devais intégrer mes meilleures idées dans son projet existant (`sync_oracle_postgresql_clean`).

Phase 1 – Pipeline ETL `pg_azure_pipeline` : mon premier chantier

Contexte et approche initiale

Dès le début du stage, j'ai voulu construire un pipeline d'ingestion de données complet. Mon objectif était d'extraire les données hospitalières depuis la base Oracle HM et de les charger dans PostgreSQL Azure. J'ai donc développé le projet `pg_azure_pipeline`, un pipeline ETL en Python conçu selon une architecture en couches avec injection de dépendances.

Architecture en couches et injection de dépendances

Ce pipeline ETL est construit selon une **architecture en couches** avec **injection de dépendances**. Cette approche est directement alignée avec la compétence CDA « Définir l'architecture logicielle d'une application ».

```
1 class IngestionPipeline:
2     """
3     L'Orchestrateur du pipeline ETL.
4     Responsabilité unique : coordonner le flux de données de la Source
5     vers la Destination. Les dépendances (extracteur, transformateur, loader)
6     sont injectées au constructeur – elles ne sont pas créées en interne.
7     """
8     def __init__(
9         self,
10        extractor: APIExtractor,
11        transformer: DataTransformer,
12        loader: Union[LoaderStrategy, Any]
13    ):
14        self.extractor = extractor
15        self.transformer = transformer
16        self.loader = loader
```


Le point d'entrée principal illustre l'injection de dépendances :

```
1 def run(loader_strategy: str = "copy_upsert"):
2     """
3     Point d'entrée du pipeline. Les composants sont créés puis injectés
4     dans l'orchestrateur, permettant de changer de stratégie de chargement
5     sans modifier le code du pipeline.
6     """
7     settings = config.settings
8     secrets_manager = SecretsManager(vault_url=settings.key_vault_uri)
9
10    # Récupération des secrets depuis Azure Key Vault
11    api_creds = secrets_manager.get_api_credentials()
12    pg_creds = secrets_manager.get_postgres_credentials()
13
14    # Injection des dépendances
15    extractor = APIExtractor(connection_params=api_creds)
16    transformer = DataTransformer()
17    loader = LoaderFactory.create(loader_strategy, pg_creds) # Pattern Factory
18
19    # Orchestration
20    pipeline = IngestionPipeline(extractor, transformer, loader)
21
22    for table_name, src_table_cfg in source_conf.items():
23        pipeline.run(src_table_cfg, tgt_table_cfg, table_name)
```

Le pattern Strategy pour les loaders

Un des aspects les plus intéressants du pipeline est l'utilisation du **pattern Strategy** pour les stratégies de chargement. J'ai implémenté quatre stratégies interchangeables, héritant d'une classe abstraite commune :

```
1 class LoaderStrategy(ABC):
2     """
3     Classe abstraite définissant le contrat pour toutes les stratégies de chargement.
4     Chaque stratégie implémente la méthode load() différemment, mais l'interface
5     reste identique pour l'orchestrateur.
6     """
7     @property
8     @abstractmethod
9     def strategy_name(self) -> str:
10        """Identifiant unique de la stratégie."""
11        pass
12
13    @abstractmethod
14    def load(self, df: pl.DataFrame, target_config: dict) -> LoadResult:
15        """Charge un DataFrame dans PostgreSQL."""
16        pass
```

La stratégie retenue pour la production est **copy_upsert** – elle utilise le protocole COPY de PostgreSQL pour le chargement en masse, qui est significativement plus performant que des INSERT individuels.

Le CopyLoader en détail

Voici le fonctionnement de la stratégie `copy_upsert`, la plus performante :

```
1 class CopyLoader(LoaderStrategy):
2     """
3     Chargement via le protocole COPY de PostgreSQL.
4
5     Algorithme en 4 étapes :
6     1. Créer une table temporaire avec le même schéma que la table cible
7     2. Charger les données dans la table temporaire via COPY FROM STDIN (CSV)
8     3. Fusionner la table temporaire dans la table cible via INSERT ... ON CONFLICT
9     4. Supprimer la table temporaire
10
11     Le protocole COPY est typiquement 5 à 10 fois plus rapide que des INSERT
12     individuels car il évite le parsing SQL pour chaque ligne.
13     """
14
15     def _copy_dataframe(self, engine, schema, table_name, df, columns=None):
16         """
17         Utilise le protocole COPY de PostgreSQL pour charger en masse.
18         Convertit le DataFrame en CSV en mémoire, puis utilise COPY FROM STDIN.
19         Compatible avec psycopg2 (copy_expert) et psycopg3 (copy).
20         """
21         temp_table = f"{table_name}{self.TEMP_SUFFIX}"
22         prepared_df = prepare_df_for_copy(df, columns)
23
24         col_list = ', '.join(f'"{c}"' for c in prepared_df.columns)
25         copy_sql = f'''COPY "{schema}"."{temp_table}" ({col_list})
26                     FROM STDIN WITH (FORMAT CSV, NULL '\\N')'''
27
28         # Écriture du DataFrame en CSV en mémoire (pas de fichier temporaire)
29         csv_buffer = io.StringIO()
30         prepared_df.write_csv(csv_buffer, include_header=False, null_value='\\N')
31         csv_buffer.seek(0)
32
33         # Utilisation du curseur natif pour COPY (pas via SQLAlchemy)
34         cursor.copy_expert(copy_sql, csv_buffer)
```

Le LoaderFactory

Le pattern Factory encapsule la logique de création des loaders :

```
1 class LoaderFactory:
2     """
3     Fabrique de stratégies de chargement. Permet de créer un loader
4     par son nom de stratégie, avec support d'alias pour la rétrocompatibilité.
5     """
6     _strategies = {
7         "simple_upsert": PolarsLoader,
8         "separate_insert_update": SeparateInsertUpdateLoader,
9         "copy_upsert": CopyLoader,
10        "delete_insert": DeleteInsertLoader,
11    }
12
13    @classmethod
14    def create(cls, strategy: str, connection_params: dict) -> LoaderStrategy:
15        loader_class = cls._strategies.get(strategy)
16        if not loader_class:
17            raise ValueError(f"Unknown strategy: {strategy}")
18        return loader_class(connection_params)
```

Métriques et monitoring du pipeline

Chaque exécution du pipeline produit un rapport détaillé de métriques :

```
1 from prometheus_client import Counter, Gauge, Histogram
2 from prometheus_fastapi_instrumentator import Instrumentator
3
4 # Métriques métier spécifiques
5 ROWS_RETURNED = Counter(
6     "oracle_api_rows_returned_total",
7     "Total de lignes retournées",
8     labelnames=("handler", "table")
9 )
10 ROWS_PER_REQUEST = Histogram(
11     "oracle_api_rows_per_request",
12     "Distribution du nombre de lignes par requête",
13     buckets=(0, 10, 100, 500, 1000, 5000, 10000, 50000, 100000)
14 )
15 DB_CONNECTION_STATUS = Gauge(
16     "oracle_api_database_connected",
17     "Statut de la connexion Oracle (1=connecté, 0=déconnecté)"
18 )
```

Ces métriques sont essentielles pour :

- Détecter les pics de mémoire (le `memory_peak_mb` devrait croître au fil des exécutions)
- Identifier les bottleneck (est-ce l'extraction, la transformation ou le chargement qui prend le plus de temps ?)
- Dimensionner l'infrastructure (RAM nécessaire, bande passante réseau)

Vérification d'intégrité post-chargement

Après chaque chargement, le pipeline peut vérifier que les données dans PostgreSQL correspondent bien aux données source :

```
1 @dataclass
2 class IntegrityResult:
3     """Résultat de la vérification d'intégrité post-chargement."""
4     verified: bool = False
5     rows_checked: int = 0
6     rows_matched: int = 0
7     rows_mismatched: int = 0
8     rows_missing: int = 0
9     mismatched_details: list = field(default_factory=list)
10     missing_pks: list = field(default_factory=list)
11     duration: float = 0.0
```

Cette vérification compare les données chargées avec les données source en comparant les hash des lignes par clé primaire. Si des divergences sont détectées, le pipeline signale l'erreur et ne met pas à jour le curseur `last_updated`, ce qui forcera un rechargement au prochain cycle.

Limites identifiées

Ce pipeline fonctionnait, mais il présentait encore un **couplage implicite** avec la source de données : l'extracteur devait connaître le format de réponse de l'API, et le pipeline dans son ensemble dépendait de la disponibilité de cette API. Ces constats, combinés aux difficultés rencontrées avec Meltano (phase suivante), m'ont conduit à développer l'API Oracle pour formaliser et stabiliser cette couche d'extraction.

Phase 2 – Meltano et Oracle Data API : deux pistes en parallèle

Cette phase regroupe deux travaux menés quasi simultanément. J'ai exploré **Meltano** comme alternative déclarative à mon pipeline custom, et c'est précisément l'impossibilité de faire fonctionner le connecteur Oracle de Meltano qui m'a conduit à développer l'**Oracle Data API**.

2a – Le pivot Meltano

Meltano est un framework open-source de pipelines de données basé sur l'écosystème **Singer** (taps et targets). L'idée est séduisante : décrire le pipeline en YAML plutôt qu'en code Python, et bénéficier de connecteurs préconstruits.

Mon premier réflexe a été de chercher un connecteur Oracle pour Meltano. Le `tap-oracle` existant s'est avéré incompatible avec ma configuration (version d'Oracle, types de données spécifiques comme les LOBs). Après plusieurs tentatives infructueuses pour le faire fonctionner, j'ai pivoté vers le `tap-rest-api-msdk`, un connecteur REST générique. Mais ce connecteur nécessitait une **API REST** comme source – d'où la naissance de l'Oracle Data API.

Configuration Meltano

Voici un extrait de la configuration Meltano que j'ai mise en place :

```
1 plugins:
2   extractors:
3     - name: tap-rest-api-msdk
4       config:
5         api_url: http://localhost:8000
6         next_page_token_path: $.pagination.next_cursor
7         streams:
8           - name: pms_rpu
9             path: /api/pms_rpu
10            primary_keys:
11              - TRPU_ID_TRPU
12            records_path: $.pms_rpu[*]
13            replication_key: TRPU_DAT_MOD
14            source_search_field: updated_at
15            source_search_query: $last_run_date
16            start_date: "1980-01-01T00:00:00+00:00"
17            schema: schemas/pms_rpu.json
18
19   loaders:
20     - name: target-postgres
21       variant: meltanolabs
22       config:
23         default_target_schema: SandBox
24         use_copy: true
25         batch_size_rows: 10000
26         load_method: upsert
27         stream_maps:
28           pms_rpu:
29             __alias__: PMS_RPU # Mapping du nom du stream vers le nom de table PG
```

Difficultés rencontrées avec Meltano

L'expérience avec Meltano s'est avérée **chronophage pour un résultat incertain**. Plusieurs difficultés m'ont freiné :

1. **Courbe d'apprentissage abrupte** : La documentation des Singer taps est complexe et fragmentée. Chaque tap a ses propres conventions, et la documentation officielle de Meltano ne couvre pas toujours les cas d'usage avancés.
2. **Échec du connecteur Oracle** : Le `tap-oracle` ne fonctionnait pas avec notre base HM. Les types de données spécifiques d'Oracle (LOBs, NUMBER avec précision variable) n'étaient pas correctement gérés

par le connecteur. C'est ce blocage qui m'a poussé à développer l'API Oracle comme couche d'abstraction.

3. **Tentative de modification du connecteur REST API pour `is_sorted`** : J'ai tenté de modifier le connecteur `tap-rest-api-msdk` pour exploiter le paramètre `is_sorted`. Ce paramètre, lorsqu'il est activé, permet au tap de reprendre l'extraction après un échec en garantissant que les données de la source sont triées chronologiquement. L'API Oracle retournait les données triées par date de modification, ce qui rendait cette optimisation théoriquement possible. En pratique, la modification du connecteur s'est avérée complexe : le code du SDK Singer est fortement abstrait, et comprendre la mécanique interne de gestion d'état m'aurait demandé un investissement disproportionné.
4. **Gestion des erreurs opaque** : Lorsque le pipeline échouait (timeout, erreur de format), les messages d'erreur de Meltano/Singer étaient souvent peu informatifs, rendant le débogage laborieux.
5. **Rigidité du YAML** : La configuration déclarative, avantage théorique de Meltano, devenait un frein dès que je devais implémenter une logique métier spécifique (transformation des BLOBs, gestion des types Oracle complexes).

Après plusieurs jours d'exploration, j'ai décidé de mettre Meltano en pause et de revenir au pipeline Python. Le pipeline Meltano reste dans le dépôt (`pg-meltano/`) comme preuve de l'exploration et comme référence si l'équipe souhaite le reprendre.

2b – Oracle Data API : extraction via FastAPI

Comme expliqué ci-dessus, l'Oracle Data API est née d'une nécessité pratique : **je n'arrivais pas à faire fonctionner le connecteur Oracle de Meltano**, et le `tap-rest-api-msdk` avait besoin d'une API REST comme source. J'ai donc développé cette API avec FastAPI pour exposer les données Oracle via des endpoints HTTP.

Mais au-delà de Meltano, cette API a rapidement montré sa valeur intrinsèque : elle **découple** complètement le pipeline de la base Oracle de production. N'importe quel client – Meltano, mon pipeline custom, dlt, ou un simple script – peut consommer les données sans avoir besoin du client Oracle installé localement.

L'API est construite autour du concept de **configuration par schéma JSON** : pour ajouter une nouvelle table, il suffit de créer un fichier JSON dans le répertoire `schemas/`, sans modifier une seule ligne de code.

Voici la structure d'un fichier de schéma :

```
1 {
2   "PMS_RPU": {
3     "schema": "SandBox",
4     "table": "PMS_RPU",
5     "last_updated": "2026-02-02 14:48:04",
6     "columns": [
7       {
8         "name": "TRPU_ID_TRPU",
9         "type": "NUMBER",
10        "length": 19,
11        "scale": 0,
12        "nullable": false,
13        "enabled": true
14      },
15      {
16        "name": "TRPU_DAT_ENT",
17        "type": "DATE",
18        "nullable": false,
19        "length": 6,
20        "scale": 0,
21        "enabled": true
22      }
23    ],
24    "keys": ["TRPU_ID_TRPU"],
25    "date_column": "TRPU_DAT_MOD"
26  }
27 }
```

Sécurité : protection contre l'injection SQL

Un aspect critique de l'API est la **protection contre l'injection SQL**. Puisque les noms de tables et de colonnes sont construits dynamiquement à partir des requêtes HTTP, j'ai implémenté une validation stricte :

```
1 import re
2 from fastapi import HTTPException
3
4 # Pattern de validation : seuls les caractères alphanumériques et underscores sont autorisés
5 VALID_IDENTIFIER = re.compile(r'^[A-Za-z_][A-Za-z0-9_]*$')
6
7 def validate_identifiant(name: str, identifiant_type: str = "identifiant") -> str:
8     """
9     Valide un identifiant SQL pour prévenir l'injection.
10    Seuls les caractères alphanumériques et underscores sont autorisés.
11    La longueur est limitée à 128 caractères.
12
13    Raises:
14        HTTPException 400 si la validation échoue
15    """
16    if not VALID_IDENTIFIER.match(name):
17        raise HTTPException(
18            status_code=400,
19            detail=f"Invalid {identifiant_type}: {name}. "
20                f"Only alphanumeric and underscore allowed."
21        )
22    if len(name) > 128:
23        raise HTTPException(
24            status_code=400,
25            detail=f"{identifiant_type} too long (max 128 characters)"
26        )
27    return name
```

Cette validation est appliquée systématiquement sur tous les identifiants SQL : noms de tables, noms de colonnes, valeurs de curseur. C'est une mesure de défense en profondeur qui complète l'utilisation de requêtes paramétrées.

Gestion des secrets avec Azure Key Vault

Les identifiants de connexion à la base Oracle ne sont jamais stockés dans le code ni dans des fichiers de configuration. Ils sont récupérés dynamiquement depuis **Azure Key Vault** :

```
1 from azure.identity import DefaultAzureCredential
2 from azure.keyvault.secrets import SecretClient
3
4 def get_keyvault_secrets() -> dict:
5     """
6     Récupère les secrets de connexion Oracle depuis Azure Key Vault.
7     Utilise DefaultAzureCredential pour l'authentification, ce qui permet
8     de fonctionner aussi bien en local (via Azure CLI) qu'en production
9     (via Managed Identity).
10    """
11    credential = DefaultAzureCredential()
12    client = SecretClient(
13        vault_url=f"https://{AZURE_KEY_VAULT_NAME}.vault.azure.net/",
14        credential=credential
15    )
16
17    secrets = {
18        "oracle_host": client.get_secret("HOSTHM-2").value,
19        "oracle_port": client.get_secret("PORTHM").value,
20        "oracle_sid": client.get_secret("SERVICEHM").value,
21        "oracle_user": client.get_secret("LOGINHM").value,
22        "oracle_password": client.get_secret("PASSWORDHM").value,
23    }
24
25    if not all(secrets.values()):
26        raise ValueError("Missing one or more required Oracle secrets from Key Vault")
27
28    return secrets
```

L'utilisation de `DefaultAzureCredential` est une bonne pratique recommandée par Microsoft : elle tente plusieurs méthodes d'authentification dans l'ordre (Managed Identity en production, Azure CLI en développement local), ce qui rend le code portable entre les environnements.

Pool de connexions et gestion des LOBs

La connexion à Oracle utilise un **pool de connexions** pour optimiser les performances :

```
1 class OracleConnection:
2     """
3     Gestionnaire de connexion Oracle avec pool de connexions.
4     Le pool maintient entre 2 et 10 connexions actives,
5     évitant le coût de création d'une connexion à chaque requête.
6     """
7     def initialize(self):
8         self.secrets = get_keyvault_secrets()
9         self.dsn = oracledb.makedsn(
10             self.secrets["oracle_host"],
11             self.secrets["oracle_port"],
12             service_name=self.secrets["oracle_sid"]
13         )
14         # Pool avec min=2, max=10 connexions
15         self.pool = oracledb.create_pool(
16             user=self.secrets["oracle_user"],
17             password=self.secrets["oracle_password"],
18             dsn=self.dsn,
19             min=2, max=10, increment=1
20         )
21
22     def create_output_type_handler(self):
23         """
24         Handler de type de sortie pour optimiser la lecture des LOBs.
25         Par défaut, oracledb retourne les BLOBs comme des objets LOB
26         nécessitant un appel .read(). Ce handler les convertit directement
27         en types natifs Python pour éviter cette étape supplémentaire.
28         """
29         def output_type_handler(cursor, metadata):
30             if metadata.type_code == oracledb.DB_TYPE_BLOB:
31                 return cursor.var(oracledb.DB_TYPE_LONG_RAW, arraysize=cursor.arraysize)
32             if metadata.type_code == oracledb.DB_TYPE_CLOB:
33                 return cursor.var(oracledb.DB_TYPE_LONG, arraysize=cursor.arraysize)
34             return output_type_handler
```

Le `output_type_handler` est un détail technique important : sans lui, chaque BLOB retourné par Oracle serait un objet LOB nécessitant un appel réseau supplémentaire (`.read()`) pour récupérer le contenu. En le convertissant directement en `LONG_RAW`, les données binaires sont récupérées en une seule opération.

Pagination par curseur (keyset pagination)

L'endpoint principal de l'API implémente une **pagination par curseur** plutôt que la pagination classique `OFFSET/LIMIT` :

```
1 @router.get("/tables/{table_name}", tags=["Data"])
2 async def get_table_data(
3     table_name: str,
4     limit: int = Query(10000, ge=1, le=100000),
5     cursor: Optional[str] = Query(None, description="Date cursor (ISO 8601)"),
6     cursor_pk: Optional[List[str]] = Query(None, description="Primary key cursor"),
7     start_time: Optional[str] = Query(None, description="Filter: date >= value"),
8     end_time: Optional[str] = Query(None, description="Filter: date <= value"),
9     response_format: str = Query("json", alias="format"),
10 ):
11     """
12     Récupère les données d'une table Oracle avec pagination et filtrage.
13
14     La pagination par curseur utilise la combinaison (date, clé_primaire)
15     pour garantir des performances constantes même sur des millions de lignes.
16     Contrairement à OFFSET/LIMIT, la complexité ne dépend pas de la position
17     dans le jeu de données.
18     """
```

La requête SQL en question aurait donc cette apparence :

```

1  SELECT pkey_column,
2         date_column
3  FROM table_name
4  WHERE date_column >= :cursor
5         OR (
6         date_column = :cursor
7         AND pkey_column > :cursor_pk
8         )
9  ORDER BY date_column,
10         pkey_column
11 FETCH FIRST :limit ROWS ONLY

```

Cette requête paramétrée implémente la **pagination par curseur** (keyset pagination). Le bloc WHERE est la clé de cette technique :

- `date_column >= :cursor` : récupère toutes les lignes dont la date est **strictement postérieure** au curseur actuel.
- `OR (date_column = :cursor AND pkey_column > :cursor_pk)` : pour les lignes ayant **exactement la même date** que le curseur, on utilise la clé primaire comme discriminant secondaire. Seules les lignes avec une clé primaire supérieure à `cursor_pk` sont incluses.

Cette logique garantit qu'aucune ligne n'est oubliée ni dupliquée entre deux pages, même si plusieurs lignes partagent la même date de modification. La combinaison (date, clé_primaire) forme un curseur unique et ordonné.

Avantage par rapport à OFFSET/LIMIT : la performance reste constante quelle que soit la profondeur de pagination (la 1000ème page est aussi rapide que la 1ère), car la base de données utilise l'index sur (date_column, pkey_column) pour localiser directement le point de départ.

Récupération des valeurs cursor et cursor_pk :

Le client récupère ces valeurs depuis la réponse de la requête précédente. À chaque page retournée, l'API inclut dans l'objet meta les valeurs cursor (dernière date de modification) et cursor_pk (dernière clé primaire) de la dernière ligne de la page. Ces valeurs sont également préconstruites dans le lien `links.next`. Pour obtenir la page suivante, le client peut soit suivre directement ce lien, soit extraire les valeurs de meta et les passer en paramètres de requête :

```

1 {
2   "data": [
3     /* ... lignes ... */
4   ],
5   "meta": {
6     "limit": 10000,
7     "row_count": 10000,
8     "has_more": true,
9     "format": "json",
10    "cursor": "2025-01-15T10:30:00",
11    "cursor_pk": 12345
12  },
13  "links": {
14    "next": "/api/tables/PMS_RPU?limit=10000&cursor=2025-01-15T10:30:00&cursor_pk=12345"
15  }
16 }

```

Observabilité avec Prometheus

L'API est instrumentée avec **Prometheus** pour le monitoring en production :


```

1 from prometheus_client import Counter, Gauge, Histogram
2 from prometheus_fastapi_instrumentator import Instrumentator
3
4 # Métriques métier spécifiques
5 ROWS_RETURNED = Counter(
6     "oracle_api_rows_returned_total",
7     "Total de lignes retournées",
8     labelnames=("handler", "table")
9 )
10 ROWS_PER_REQUEST = Histogram(
11     "oracle_api_rows_per_request",
12     "Distribution du nombre de lignes par requête",
13     buckets=(0, 10, 100, 500, 1000, 5000, 10000, 50000, 100000)
14 )
15 DB_CONNECTION_STATUS = Gauge(
16     "oracle_api_database_connected",
17     "Statut de la connexion Oracle (1=connecté, 0=déconnecté)"
18 )

```

Ces métriques permettent de :

- Surveiller le volume de données transféré
- Détecter les anomalies (pic de requêtes, perte de connexion Oracle)
- Optimiser les tailles de page (via l'histogramme des lignes par requête)

Phase 3 – Découverte tardive de dlt (dlt-hub)

Qu'est-ce que dlt ?

dlt (*data load tool*) est une bibliothèque Python open-source qui se positionne entre le « tout coder à la main » et le « tout déclarer en YAML ». Elle fournit des classes, des helpers et des outils prêts à l'emploi pour construire un pipeline fonctionnel et sécurisé, tout en restant dans l'écosystème Python.

Implémentation avec dlt

J'ai découvert dlt tardivement dans le stage et j'ai implémenté un pipeline de test dans dlt-pipeline/ :

```

1 import dlt
2 from dlt.sources.rest_api import RESTAPIConfig, rest_api_resources
3
4 @dlt.source(name="hm_oracle_api", parallelized=True)
5 def hm_oracle_api_source(access_token=dlt.secrets.value):
6     """
7     Source dlt pour l'API Oracle HM.
8     La configuration utilise RESTAPIConfig pour le typage et l'autocomplétion.
9     Le paginateur json_link suit automatiquement les liens 'next' de l'API.
10    Le chargement incrémental est géré nativement via le cursor_path.
11    """
12    config: RESTAPIConfig = {
13        "client": {
14            "base_url": "http://localhost:8000/api/",
15            "paginator": "json_link", # Suit automatiquement links.next
16        },
17        "resource_defaults": {
18            "write_disposition": "merge", # UPSERT par défaut
19            "endpoint": {
20                "params": {"limit": 10000},
21            },
22        },
23        "resources": [
24            {
25                "name": "pms_rpu",
26                "endpoint": {
27                    "path": "pms_rpu",
28                    "params": {"limit": 20000},
29                    "paginator": {
30                        "type": "json_link",
31                        "next_url_path": "links.next",
32                    },
33                    # Chargement incrémental natif : dlt mémorise la dernière
34                    # valeur du cursor et ne récupère que les données plus récentes
35                    "incremental": {
36                        "cursor_path": "TRPU_DAT_MOD",
37                        "initial_value": pendulum.now().subtract(days=30).to_iso8601_string(),
38                    },
39                },
40            },
41            # ... autres tables ...
42        ],
43    }
44    yield from rest_api_resources(config)
45
46 def load_hm_oracle_api():
47     """Exécution du pipeline dlt vers PostgreSQL."""
48     pipeline = dlt.pipeline(
49         pipeline_name="rest_api_hm_oracle_api",
50         destination='postgres',
51         dataset_name="oracle_dlt",
52     )
53     load_info = pipeline.run(hm_oracle_api_source())
54     print(load_info)

```

Avantages de dlt constatés

Après cette implémentation, j'ai identifié plusieurs avantages significatifs de dlt par rapport aux autres approches :

- **On code en Python** : Contrairement à Meltano, il n'y a pas de couche YAML entre le développeur et la logique du pipeline. Les IDE fournissent l'autocomplétion, le typage est vérifié, et les erreurs sont des erreurs Python classiques.
- **Incrémentalité native** : dlt gère automatiquement l'état du pipeline (cursor, dernière valeur traitée) et le persiste dans un fichier d'état interne. Pas besoin de gérer manuellement un fichier last_updated.
- **Schéma automatique** : dlt infère et gère le schéma de la destination automatiquement. Il crée les tables, détecte les changements de schéma et les applique (ajout de colonnes, par exemple).
- **Parallélisation** : Le décorateur `@dlt.source(parallelized=True)` permet l'extraction parallèle de plusieurs tables.
- **Écosystème riche** : dlt fournit des sources préconstruites (`rest_api`, `sql_database`, etc.) qui accélèrent considérablement le développement.

Regret et leçon tirée

Je regrette de ne pas avoir découvert dlt plus tôt dans le stage. Si c'était à refaire, je commencerais par dlt plutôt que par un pipeline « fait maison ». Le pipeline custom m'a beaucoup appris (patterns de conception, protocole COPY, gestion de connexions), mais dlt aurait permis d'atteindre un résultat fonctionnel plus rapidement.

Cela dit, l'exercice de développer un pipeline complet de zéro a une valeur pédagogique indéniable : je comprends maintenant en profondeur les problématiques de la réplication de données (gestion d'état, gestion des erreurs, performances du chargement, intégrité des données).

Phase 4 – Intégration dans `sync_oracle_postgresql_clean` : le projet de Marine

Convergence des travaux

Après plusieurs semaines de développement en parallèle, Marine Crognier a comparé mes projets (`pg_azure_pipeline`, `oracle_api`) au sien (`sync_oracle_postgresql_clean`). Sa conclusion a été claire : plutôt que de maintenir deux pipelines séparés, je devais **intégrer mes meilleures idées dans son projet existant**. Ce projet, qu'elle avait initié avant mon arrivée avec les bibliothèques `cx_Oracle`, `SQLAlchemy` et `Pandas`, constituait le socle opérationnel reconnu par l'équipe.

Cette décision m'a permis de capitaliser sur les forces de chaque approche :

- Du projet de Marine : la connaissance métier des 84 tables, le logging bien établi, l'infrastructure déjà en place.
- De mes projets : l'architecture en couches, les stratégies de chargement (COPY UPSERT retenu), l'API REST de synchronisation, la gestion des métriques et la vérification d'intégrité.

Configuration des tables avec Pydantic

Le projet `sync_oracle_postgresql_clean` gère la configuration des 84 tables à synchroniser via un modèle Pydantic, garantissant la validation des paramètres. J'ai contribué à renforcer cette validation :

```
1 class TableConfig(BaseModel):
2     """
3     Configuration d'une table à synchroniser avec validation Pydantic.
4     Chaque table est définie par son nom, sa clé primaire, sa colonne de date
5     de modification et son schéma cible PostgreSQL.
6     """
7     name: str = Field(..., min_length=1, description="Nom de la table")
8     primary_key: str = Field(..., min_length=1, description="Clé primaire")
9     date_column: str = Field(..., min_length=1, description="Colonne de date de modification")
10    oracle_schema: Optional[str] = Field(default=None, description="Schéma Oracle")
11    postgresql_schema: str = Field(default="SandBox", description="Schéma PostgreSQL cible")
12    enabled: bool = Field(default=True, description="Table activée pour synchronisation")
13    chunk_size: Optional[int] = Field(default=None, gt=0, description="Taille de chunk personnalisée")
14    description: str = Field(default="", description="Description fonctionnelle de la table")
15    sync_columns: Optional[List[str]] = Field(
16        default=None,
17        description="Colonnes spécifiques à synchroniser (None = toutes)"
18    )
19
20    # Validation : la clé primaire doit figurer dans les colonnes à synchroniser
21    @model_validator(mode='after')
22    def validate_primary_key_in_sync_columns(self) -> 'TableConfig':
23        if self.sync_columns is not None and self.primary_key not in self.sync_columns:
24            raise ValueError(
25                f"primary_key '{self.primary_key}' must be included in sync_columns"
26            )
27        return self
```

Ce modèle illustre plusieurs bonnes pratiques :

- **Validation à la construction** : les erreurs de configuration sont détectées immédiatement grâce aux validateurs Pydantic.
- **Valeurs par défaut sensées** : le schéma PostgreSQL cible par défaut est SandBox, le chunk_size est optionnel.
- **Cohérence métier** : le validateur validate_primary_key_in_sync_columns garantit que la clé primaire sera toujours présente dans les données synchronisées.

Leçon tirée : la valeur de la collaboration

Cette phase m’a appris une leçon importante : en entreprise, dupliquer les efforts sur deux prototypes parallèles est un gaspillage. Le PoC de Marine, bien qu’imparfait, avait l’avantage de l’antériorité et d’une connaissance métier accumulée sur les 84 tables. Fusionner les forces des deux projets – sa couverture fonctionnelle et mes améliorations techniques – était plus productif que de maintenir deux bases de code concurrentes.

Phase 5 – Stratégies d’ingestion testées

Au cours du stage, j’ai testé et comparé plusieurs stratégies d’ingestion. Cette comparaison est synthétisée ci-dessous :

Tableau comparatif des stratégies d’ingestion

Stratégie	Principe	Avantages	Inconvénients	Performance
Simple UPSERT	INSERT ... ON CONFLICT DO UPDATE direct	Simple, atomique	Lent pour les grands volumes	x0.1
Separate INSERT/UPDATE	Sépare les opérations	Contrôle granulaire, métriques détaillées	Plus complexe, 2 passes	x0.2
DELETE + INSERT	Supprime puis réinsère	Garantie de données fraîches	Risque de perte en cas d’erreur	x0.8
COPY + UPSERT	COPY → temp table → upsert	Très rapide , utilise le protocole natif PG	Nécessite table temporaire	x1.0 (baseline)

La stratégie **COPY + UPSERT** a été retenue pour la production. Le protocole COPY de PostgreSQL est le mécanisme le plus efficace pour le chargement en masse : il évite le parsing SQL pour chaque ligne, utilise un format binaire ou CSV optimisé, et bénéficie d’optimisations internes de PostgreSQL.

Réflexion sur l’approche append-only + déduplication

Avec le recul, une stratégie **append-only avec déduplication** aurait été potentiellement plus performante et plus simple que l’UPSERT :

- **Append-only** : Chaque exécution du pipeline insère les nouvelles données sans vérifier les conflits. La table cible contient donc des doublons (plusieurs versions d’une même ligne).
- **Déduplication** : Une vue matérialisée ou une requête avec ROW_NUMBER() OVER (PARTITION BY pk ORDER BY date_mod DESC) extrait la dernière version de chaque ligne.

Avantages théoriques :

- Pas de lock sur la table cible (pas de ON CONFLICT)
- Insertion pure, donc vitesse maximale
- Historique complet des modifications (utile pour l’audit)

Pourquoi cette approche n’a pas été explorée :

- Manque de temps en fin de stage

- Nécessite une base de données optimisée pour les agrégations (type ClickHouse) plutôt que PostgreSQL (OLTP)
- Plus adaptée à une stack data moderne (cf. section Veille technologique)

Je documente cette réflexion ici car elle représente une **perspective d'amélioration** que l'équipe pourrait explorer.

Sécurité de l'application

La sécurité est une préoccupation transversale dans ce projet. Le REAC CDA insiste sur le développement d'applications « sécurisées », et j'ai appliqué plusieurs mesures à différents niveaux.

Protection contre l'injection SQL

Comme détaillé dans la section précédente, tous les identifiants SQL (noms de tables, colonnes) sont validés par une expression régulière avant d'être utilisés dans des requêtes. Cette validation est systématique et constitue la première ligne de défense.

De plus, les valeurs de paramètres (dates, curseurs) sont passées via des **paramètres de requête liés** (bind parameters) plutôt que concaténées dans la chaîne SQL :

```
1 # Vulnérable à l'injection SQL
2 query = f"SELECT * FROM {table} WHERE date > '{user_input}'"
3
4 # Sécurisé : paramètre lié
5 query = f"SELECT * FROM :table WHERE date > :cursor_date"
6 params = {"table": table, "cursor_date": parsed_date}
```

Gestion des secrets

Aucun secret (mot de passe, clé d'API) n'est stocké dans le code source ou dans des fichiers de configuration. Tous les secrets sont récupérés dynamiquement depuis **Azure Key Vault** via `DefaultAzureCredential`. Cette approche :

- Évite l'exposition accidentelle de secrets dans les dépôts Git
- Permet la rotation des secrets sans redéploiement
- S'adapte automatiquement à l'environnement (développement local vs production)

Rate limiting

L'API Oracle implémente un **rate limiting** via la bibliothèque `SlowAPI` :

```
1 from slowapi import Limiter
2 from slowapi.util import get_remote_address
3
4 limiter = Limiter(key_func=get_remote_address, default_limits=["1/minute"])
5
6 @router.get("/tables/{table_name}")
7 @limiter.limit("60/minute")
8 async def get_table_data(request: Request, ...):
9     ...
```

Le rate limiting protège l'API contre les abus et les surcharges accidentelles. Les limites sont configurées par endpoint : 100/minute pour le root, 60/minute pour les endpoints de données.

Compression des réponses

L'API active la compression GZip pour les réponses volumineuses :

```
1 from fastapi.middleware.gzip import GZipMiddleware
2 app.add_middleware(GZipMiddleware, minimum_size=500)
```

Cela réduit la bande passante consommée et améliore les temps de réponse pour les clients qui acceptent la compression.

Validation des entrées

Toutes les entrées utilisateur sont validées :

- Les dates sont parsées via `dateutil.parser` avec `fuzzy=False` pour rejeter les formats invalides.
- Les formats de réponse sont limités à `json` et `msgpack`.
- Les limites de pagination sont bornées (1 à 100 000).
- Les noms de colonnes sont vérifiés contre le schéma de la table.

Plan de test

Stratégie de test

J'ai adopté une approche de test à plusieurs niveaux, conformément aux bonnes pratiques du développement logiciel :

- **Tests unitaires** : Testent les fonctions individuelles en isolation (validation des identifiants, sérialisation des types Oracle, parsing des dates).
- **Tests d'intégration** : Testent les interactions entre composants (appels API avec base de données mockée, pipeline complet avec données de test).
- **Tests de bout en bout** : Testent le flux complet de données de l'API Oracle jusqu'à PostgreSQL.

Exemple de tests unitaires

```
1 # test_schemas.py - Tests de validation des identifiants SQL
2 import pytest
3 from schemas import validate_identifiant
4
5 class TestValidateIdentifiant:
6     """Tests pour la fonction de validation des identifiants SQL."""
7
8     def test_valid_identifiant(self):
9         """Un identifiant valide ne doit pas lever d'exception."""
10         assert validate_identifiant("PMS_RPU") == "PMS_RPU"
11         assert validate_identifiant("col_name_123") == "col_name_123"
12
13     def test_invalid_identifiant_with_special_chars(self):
14         """Un identifiant avec des caractères spéciaux doit être rejeté."""
15         with pytest.raises(HTTPException) as exc_info:
16             validate_identifiant("DROP TABLE; --")
17         assert exc_info.value.status_code == 400
18
19     def test_identifiant_too_long(self):
20         """Un identifiant dépassant 128 caractères doit être rejeté."""
21         long_name = "A" * 129
22         with pytest.raises(HTTPException) as exc_info:
23             validate_identifiant(long_name)
24         assert exc_info.value.status_code == 400
25
26     def test_empty_identifiant(self):
27         """Un identifiant vide doit être rejeté."""
28         with pytest.raises(HTTPException):
29             validate_identifiant("")
```



```

1 # test_serializers.py - Tests de sérialisation des types Oracle
2 class TestSerializeOracleValue:
3     """Tests pour la conversion des types Oracle vers des types sérialisables."""
4
5     def test_datetime_to_iso(self):
6         """Les datetime Oracle doivent être convertis en ISO 8601."""
7         dt = datetime(2025, 1, 15, 10, 30, 0)
8         assert serialize_oracle_value(dt) == "2025-01-15T10:30:00"
9
10    def test_blob_to_base64(self):
11        """Les BLOB doivent être encodés en base64 pour JSON."""
12        blob_data = b'\x89PNG\r\n'
13        result = serialize_oracle_value(blob_data, for_msgpack=False)
14        assert isinstance(result, str) # base64 string
15
16    def test_blob_raw_for_msgpack(self):
17        """Les BLOB doivent rester en bytes pour msgpack."""
18        blob_data = b'\x89PNG\r\n'
19        result = serialize_oracle_value(blob_data, for_msgpack=True)
20        assert isinstance(result, bytes)
21
22    def test_decimal_integer(self):
23        """Les Decimal sans décimales doivent être convertis en int."""
24        from decimal import Decimal
25        assert serialize_oracle_value(Decimal("42")) == 42
26
27    def test_none_value(self):
28        """Les valeurs None doivent rester None."""
29        assert serialize_oracle_value(None) is None

```

Couverture de tests

Les tests couvrent les modules suivants :

- `schemas.py` : Validation des identifiants, chargement des schémas JSON
- `serializers.py` : Sérialisation des types Oracle (datetime, BLOB, Decimal)
- `routes/data.py` : Endpoint de données (pagination, filtrage)
- `routes/health.py` : Endpoint de santé
- `src/loaders/` : Stratégies de chargement (COPY, UPSERT)
- `src/transform.py` : Transformations de données
- `src/config.py` : Gestion de la configuration et mise à jour du curseur

Architecture de déploiement cible

Note importante : Ce plan de déploiement est **théorique**. Par manque de temps, je n'ai pas pu l'étudier en profondeur ni le tester en conditions réelles. Il représente une ébauche basée sur les connaissances et les intentions de la cheffe de projet et la documentation Azure, mais devra être validé et affiné par l'équipe avant toute mise en production.

Procédure de déploiement de l'API Oracle

1. Préparation de l'environnement :

```
1 # Créer l'environnement virtuel
2 python -m venv .fastapi
3 .\.fastapi\Scripts\Activate.ps1
4
5 # Installer les dépendances
6 pip install -r requirements.txt
7
8 # Installer Oracle Instant Client 23.6
9 # Configurer le PATH vers C:\oracle\instantclient_23_6
```

2. Configuration Azure :

```
1 # Authentification Azure (pour Key Vault)
2 az login
3
4 # Vérifier l'accès au Key Vault
5 az keyvault secret list --vault-name kv-regul-ia-prod
```

3. Lancement de l'API :

```
1 # Mode développement (avec hot-reload)
2 uvicorn main:app --reload --host 0.0.0.0 --port 8000
3
4 # Mode production (avec workers multiples)
5 uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4
```

4. Vérification :

```
1 # Test de santé
2 curl http://localhost:8000/health
3
4 # Liste des tables configurées
5 curl http://localhost:8000/api/tables
```

Pipeline Azure DevOps (CI/CD)

Un fichier `azure-pipelines.yml` a été préparé pour l'intégration continue :

```
1 trigger:
2   - main
3
4 pool:
5   vmImage: "ubuntu-latest"
6
7 steps:
8   - task: UsePythonVersion@0
9     inputs:
10       versionSpec: "3.10"
11
12   - script: |
13       pip install -r requirements.txt
14       pip install pytest pytest-cov
15       displayName: "Install dependencies"
16
17   - script: |
18       pytest tests/ --cov=. --cov-report=xml
19       displayName: "Run tests"
20
21   - task: PublishTestResults@2
22     inputs:
23       testResultsFiles: "**/test-results.xml"
```

Conformité et Responsabilité Numérique

En tant que Concepteur Développeur d'Applications, ma responsabilité ne se limite pas à produire du code fonctionnel. Elle s'étend à la protection des utilisateurs et à l'impact environnemental de mes applications. J'ai pris en compte ces deux piliers, le RGPD et l'Éco-conception, dans ce projet.

1. Respect du Règlement Général sur la Protection des Données (RGPD)

Le REAC stipule que le développeur « met en place les mentions légales liées au règlement général sur la protection des données (RGPD) » et « respecte le règlement général sur la protection des données (RGPD) en vigueur ».

Dans le contexte hospitalier, la sensibilité des données est maximale. Les données traitées incluent :

- Des identifiants de patients (pseudonymisés dans la base Oracle)
- Des dates de passages aux urgences
- Des diagnostics médicaux (codes CIM-10)
- Des informations de séjour

Mesures appliquées :

- **Minimisation des données** : Seules les colonnes nécessaires à l'analyse sont synchronisées. Le paramètre `sync_columns` dans la configuration des tables permet d'exclure les colonnes non nécessaires. Par exemple, pour la table `BAS_DOCUMENT`, la colonne `DOCU_BLOB_CONTENU` (contenu binaire des documents) est explicitement exclue.
- **Chiffrement en transit** : Les connexions à PostgreSQL Azure utilisent SSL/TLS obligatoire en production. Les connexions à Azure Key Vault sont sécurisées par HTTPS.
- **Chiffrement au repos** : PostgreSQL Azure chiffre les données au repos par défaut (AES-256) en production.
- **Contrôle d'accès** : L'accès à la base PostgreSQL est contrôlé par Azure Active Directory. L'accès aux secrets est limité via les politiques d'accès de Key Vault.
- **Pas de données nominatives en logs** : Les logs du pipeline ne contiennent jamais de données patients. Seuls les noms de tables, les compteurs de lignes et les métriques de performance sont journalisés.

2. Principes d'Éco-conception (Green IT)

Le REAC m'amène à « identifier les besoins en éco-conception » et à « appliquer les recommandations concernant l'écoconception des services numériques ». J'ai appliqué cette démarche à plusieurs niveaux :

- **Transfert optimisé** : Le support de MessagePack (~25% plus compact que JSON) réduit le volume de données transféré sur le réseau. La compression GZip réduit également la bande passante.
- **Incrémentalité** : Ne transférer que les données modifiées plutôt que la totalité des tables réduit considérablement la consommation de ressources (CPU, réseau, stockage).
- **Protocole COPY** : Utiliser le protocole natif COPY de PostgreSQL plutôt que des INSERT individuels réduit le nombre d'allers-retours réseau et la consommation CPU du serveur de base de données.
- **Pool de connexions** : Le pool de connexions Oracle (min=2, max=10) évite la création/destruction répétée de connexions, réduisant la charge serveur.

- **Polars plutôt que Pandas** : Polars consomme significativement moins de mémoire que Pandas pour des volumes équivalents, grâce au format Apache Arrow en colonne.

Difficultés rencontrées et exploration

Cette section est volontairement détaillée car le REAC CDA évalue la capacité du candidat à analyser et surmonter les difficultés. Chaque difficulté est présentée avec son contexte, son impact et la solution ou leçon tirée.

Difficultés liées à Meltano et Singer

Courbe d'apprentissage

Meltano repose sur l'écosystème Singer, un standard de connecteurs de données composé de « taps » (extracteurs) et « targets » (loaders). La documentation de ces connecteurs est dispersée entre plusieurs sources : la documentation officielle de Meltano, la documentation du SDK Singer, les README des connecteurs individuels sur GitHub, et les discussions communautaires.

J'ai passé plusieurs jours à comprendre les concepts de base : les catalogues, les schémas, la gestion d'état (state), les mécanismes de réplication (full table, incremental, log-based). Cette immersion, bien que formatrice, a ralenti la progression du projet.

Tentative de modification du connecteur REST API

Le connecteur `tap-rest-api-msdk` que j'utilisais supporte le paramètre `is_sorted`, qui indique que les données de la source sont déjà triées par la `replication_key` (dans notre cas, la date de modification). Lorsque ce paramètre est activé, le tap peut :

- Mémoriser la dernière valeur de la `replication_key` traitée
- En cas d'échec, reprendre l'extraction à partir de cette valeur
- Garantir qu'aucune donnée n'est perdue ni dupliquée

Mon API Oracle retournait bien les données triées par `TRPU_DAT_MOD`, ce qui rendait cette optimisation théoriquement applicable. Cependant, en pratique :

- Le code du SDK Singer qui gère `is_sorted` était profondément imbriqué dans la logique de gestion d'état.
- Modifier ce comportement nécessitait de comprendre le mécanisme de `state messages` et de `bookmarks` du protocole Singer.
- Les tests du connecteur n'étaient pas facilement reproductibles en local sans un environnement Singer complet.

Après deux jours d'exploration, j'ai abandonné cette piste. L'investissement nécessaire était disproportionné par rapport au gain attendu, d'autant que mon pipeline Python gérait déjà l'incrémentalité de manière fiable.

Constat final sur Meltano

Meltano est un outil puissant pour les équipes data qui l'utilisent quotidiennement et qui ont investi dans sa maîtrise. Pour un développeur qui découvre l'écosystème Singer, la courbe d'apprentissage est raide et le retour sur investissement n'est pas garanti à court terme. L'approche YAML-first est séduisante en théorie, mais devient un frein dès que les besoins sortent du cadre standard.

Découverte tardive de dlt

Le meilleur compromis

La bibliothèque `dlt` représente à mon avis le **meilleur compromis** entre les trois approches testées :

Critère	Pipeline custom	Meltano	dlt
Langage	Python pur	YAML + Singer	Python pur
Flexibilité	Totale	Limitée par les connecteurs	Élevée
Courbe d'apprentissage	Moyenne (si on connaît Python)	Élevée	Faible
Fonctionnalités intégrées	Aucune (tout à coder)	Nombreuses (connecteurs, UI)	Nombreuses (schéma auto, état, retry)
Maintenance	Élevée	Moyenne	Faible
Tests	À écrire entièrement	Dépend des connecteurs	Facilités par les helpers dlt

Regret

Je regrette sincèrement de ne pas avoir découvert dlt plus tôt dans le stage. Si je devais recommencer le projet, ma stratégie serait :

- **Semaine 1** : Développer l'API Oracle (incontournable, car elle découple le pipeline de la base de production)
- **Semaine 2-3** : Utiliser dlt pour le pipeline d'ingestion, en profitant de ses fonctionnalités intégrées (pagination, incrémentalité, schéma auto)
- **Semaine 4+** : Se concentrer sur la qualité (tests, monitoring, documentation) plutôt que sur la plomberie technique

Le temps investi dans Meltano et dans le développement du pipeline custom aurait pu être redirigé vers des aspects à plus forte valeur ajoutée.

Difficultés organisationnelles

Nouveau domaine

Le data engineering était un domaine entièrement nouveau pour moi. Mes compétences initiales étaient orientées développement d'application (Front-end et Back-end). La transition vers le data engineering a nécessité l'apprentissage de :

- Les concepts ETL/ELT
- La gestion d'état pour les pipelines incrémentaux
- Les protocoles de chargement en masse (COPY, bulk insert)
- L'écosystème Singer/Meltano
- Les spécificités d'Oracle Database (LOBs, types de données)
- L'infrastructure Azure (Key Vault, PostgreSQL Azure)

Gestion du temps

Avec le recul, j'aurais dû consacrer plus de temps à la phase de cadrage technique initiale. L'approche « plonger et explorer » que j'ai adoptée m'a fait perdre du temps sur des pistes infructueuses (Meltano notamment). Un sprint initial de quelques jours consacré à un état de l'art des outils d'ingestion Python (comparant dlt, Meltano, Airbyte, un pipeline custom) m'aurait permis de faire un choix éclairé dès le départ.

Cette leçon est précieuse pour mes futurs projets : **le temps investi dans la recherche en amont est rarement du temps perdu.**

Veille technologique – Stack data moderne

Cette section présente les résultats de ma veille technologique sur les outils de la « stack data moderne ». Ces recommandations constituent une ouverture pour l'équipe du Centre Hospitalier, au-delà du périmètre de mon stage.

Stack recommandée

Outil	Rôle	Avantage clé
dlt	ELT/ETL en Python	Plus flexible que Meltano, plus simple qu'Airbyte. On reste dans l'écosystème Python avec des helpers prêts à l'emploi.
SQLMesh	Transformation	Alternative moderne à dbt. Gestion native des environnements virtuels, tests intégrés, plan de transformation visuel.
ClickHouse	Base OLAP	Optimisé pour les agrégations (GROUP BY) vs PostgreSQL (OLTP). Support natif de l'append-only + déduplication. Plus de stockage mais moins de RAM.
Dagster	Orchestration	<i>Data-asset-aware</i> nativement (vs Airflow < 3.0, orienté procédures). Chaque asset est un nœud du graphe, avec lignage automatique.

dlt – Data Load Tool

La bibliothèque dlt est la pièce maîtresse de cette stack. Comme démontré dans la section précédente, il combine la flexibilité du code Python avec des abstractions de haut niveau pour les tâches répétitives (pagination, schéma, état, retry). Pour l'équipe du Centre Hospitalier, dlt permettrait de :

- Ajouter de nouvelles sources de données (autres API, fichiers CSV, bases de données) sans repartir de zéro
- Bénéficier de la gestion automatique du schéma (ajout de colonnes, évolution du schéma source)
- Réduire la dette technique liée au pipeline custom

SQLMesh – Transformation

SQLMesh est une alternative moderne à **dbt** (data build tool). Il permet de définir des transformations SQL ou Python, avec des fonctionnalités avancées :

- **Environnements virtuels** : tester les transformations sans impacter la production
- **Plans de transformation** : visualiser les changements avant de les appliquer
- **Tests intégrés** : définir des assertions directement dans les modèles

Dans notre contexte, SQLMesh pourrait être utilisé pour :

- Créer des vues agrégées (nombre de passages aux urgences par jour, par type de diagnostic, etc.)
- Anonymiser ou pseudonymiser les données pour les analystes
- Calculer des indicateurs de performance hospitalière

ClickHouse – Base OLAP

ClickHouse est une base de données orientée colonnes, optimisée pour les requêtes analytiques. Contrairement à PostgreSQL (qui est un SGBD OLTP, optimisé pour les transactions individuelles), ClickHouse excelle dans les agrégations sur de grands volumes de données.

Pourquoi ClickHouse serait pertinent ici :

- Les requêtes analytiques sur les données des urgences sont principalement des agrégations (COUNT, AVG, GROUP BY) – exactement le cas d’usage de ClickHouse.
- ClickHouse supporte nativement la stratégie **append-only + déduplication** via les moteurs de table ReplacingMergeTree. Chaque insertion est un simple ajout ; la déduplication est gérée en arrière-plan.
- Cela élimine le besoin de l’UPSERT (qui pose des problèmes de lock et de performance sur PostgreSQL pour les gros volumes).

Limitation : ClickHouse n’est pas disponible en service managé Azure en France au moment de la rédaction. Cependant, l’auto-hébergement est réputé simple (un seul binaire, configuration minimale). L’équipe pourrait envisager un déploiement sur une VM Azure ou un conteneur.

Comparaison append-only + déduplication vs UPSERT :

Aspect	UPSERT (PostgreSQL)	Append-only + dédup (ClickHouse)
Écriture	Lock sur les lignes en conflit	Ajout simple, pas de lock
Lecture	Donnée toujours à jour	Nécessite déduplication à la lecture
Historique	Perdu (écrasement)	Conservé (toutes les versions)
Performance écriture	Modérée	Très élevée
Stockage	Optimisé (une seule version)	Plus élevé (toutes les versions)
RAM	Plus élevée (index B-tree)	Plus faible (compression colonnaire)

Dagster – Orchestration

Dagster est un orchestrateur de pipelines de données qui se distingue d’Apache Airflow par son approche centrée sur les **assets** (données) plutôt que sur les **tâches** (procédures) :

- Dans Airflow (< 3.0), on définit un DAG de tâches : « faire ceci, puis cela ».
- Dans Dagster, on définit des assets : « cette table dépend de ces sources ». L’orchestrateur en déduit automatiquement l’ordre d’exécution.

Cette approche est plus naturelle pour le data engineering, où l’on pense en termes de « quelles données dois-je produire ? » plutôt que « quelles étapes dois-je exécuter ? ».

Dagster s’intègre nativement avec dlt, ce qui permettrait de construire une stack cohérente : **dlt** pour l’ingestion, **SQLMesh** pour la transformation, **Dagster** pour l’orchestration.

Bilan

Apports et apprentissages

Ce stage m'a apporté des compétences significatives dans plusieurs domaines :

Compétences techniques acquises

- **Data engineering** : J'ai acquis une compréhension approfondie des problématiques de réplication de données : gestion d'état, incrémentalité, stratégies de chargement, intégrité des données, gestion des types complexes (LOBs, Decimal).
- **Architecture logicielle** : J'ai mis en pratique les patterns de conception (Strategy, Factory, injection de dépendances) dans un contexte réel, avec des bénéfices concrets (interchangeabilité des loaders, testabilité).
- **API REST** : J'ai développé deux API REST complètes avec FastAPI, incluant la pagination, le rate limiting, le monitoring Prometheus et la documentation OpenAPI automatique.
- **Sécurité** : J'ai implémenté des mesures de sécurité concrètes (validation des identifiants SQL, gestion des secrets via Key Vault, chiffrement en transit).
- **PostgreSQL** : J'ai découvert le protocole COPY, les contraintes d'UPSERT, et les considérations de performance pour le chargement en masse.
- **Oracle Database** : J'ai appris à gérer les spécificités d'Oracle (LOBs, output_type_handler, pool de connexions).
- **Écosystème Azure** : J'ai utilisé Azure Key Vault, PostgreSQL Azure, et Azure DevOps.
- **Outils data** : J'ai exploré Meltano, dlt, Polars, et j'ai mené une veille sur ClickHouse, SQLMesh et Dagster.

Compétences transverses

- **Communication** : Les échanges quotidiens avec Marine Crognier via Teams m'ont appris à communiquer efficacement dans un contexte professionnel, à formuler des questions précises et à présenter mes avancements de manière structurée.
- **Autonomie** : J'ai dû apprendre un domaine entièrement nouveau de manière autonome, en m'appuyant sur la documentation, les tutoriels et l'expérimentation.
- **Recul critique** : Ce stage m'a appris l'importance du cadrage technique en amont d'un projet. L'exploration non structurée peut être formatrice mais coûteuse en temps.

Difficultés surmontées

- **Meltano** : Après plusieurs jours d'exploration infructueuse, j'ai su reconnaître que cette piste n'était pas productive et j'ai pivoté vers une solution plus pragmatique.
- **Optimisation de l'API Oracle** : Les premières versions de l'API souffraient de performances médiocres sur les grandes tables. J'ai dû implémenter plusieurs optimisations : le pool de connexions pour éviter la création répétée de connexions coûteuses, le output_type_handler pour la lecture efficace des LOBs, la pagination par curseur (keyset) pour garantir des performances constantes quelle que soit la profondeur de pagination et l'encodage base64 pour JSON, bytes brutes pour MessagePack, encodage hex pour COPY CSV.
- **Optimisation de l'ingestion PostgreSQL** : Le chargement initial via des INSERT individuels était beaucoup trop lent pour les tables volumineuses (plus de 5 millions de lignes pour certaines). J'ai testé et comparé quatre stratégies de chargement avant de converger vers le protocole COPY combiné à l'UPSERT, offrant un gain de performance de 5 à 10x par rapport aux INSERT classiques.

Points d'amélioration identifiés

- **Organisation** : J'aurais dû consacrer plus de temps au cadrage technique initial pour éviter les détours par Meltano.
- **Tests** : La couverture de tests, bien que correcte, pourrait être améliorée. Les tests de bout en bout sont notamment insuffisants.
- **Documentation** : Certains modules mériteraient une documentation plus détaillée, notamment les stratégies de chargement et la gestion des erreurs.

Perspectives

- Pour la suite de ce projet et pour d'éventuelles améliorations, je recommande :
- **Adopter dlt** comme outil principal d'ingestion, en remplacement du pipeline custom. dlt réduirait significativement la dette technique et faciliterait l'ajout de nouvelles sources.
- **Explorer ClickHouse** comme base OLAP de destination, en complément de PostgreSQL. L'approche append-only + déduplication serait plus performante et plus simple que l'UPSERT pour les volumes importants.
- **Mettre en place Dagster** pour l'orchestration, afin d'automatiser les exécutions du pipeline et de surveiller les assets de données.
- **Ajouter des transformations** avec SQLMesh pour produire des vues agrégées directement exploitables par les analystes.
- **Poursuivre les tests** pour atteindre une couverture de code proche de 100%.
- **Optimiser le code** pour améliorer la scalabilité, notamment la gestion mémoire pour les très grandes tables.
- **Ajouter l'internationalisation** (i18n) et la localisation (l10n) à l'API pour une éventuelle utilisation dans d'autres établissements.

Remerciements

Au terme de ce projet de stage, je tiens à exprimer mes sincères remerciements.

Je remercie tout d'abord l'organisme de formation **2iSA** pour la qualité de l'enseignement dispensé et les moyens mis à disposition pour notre apprentissage.

Ma gratitude va ensuite tout particulièrement à **Marine Crognier**, ma maîtresse de stage, son expertise du domaine hospitalier et ses conseils avisés. Son travail préparatoire sur la synchronisation Oracle → PostgreSQL a constitué un socle solide sur lequel j'ai pu construire mes développements. Ses retours réguliers et bienveillants m'ont permis de progresser et de garder le cap malgré les difficultés techniques rencontrées.

Je remercie également mes formateurs **Serge, Fabien et Théo** pour leur accompagnement constant, leur disponibilité et leurs précieux conseils tout au long de cette période, qui ont été déterminants dans ma progression et m'ont encouragé à persévérer.

Je souhaite remercier mes collègues de promotion pour l'entraide, les échanges constructifs et l'excellente ambiance de travail qui ont grandement contribué à rendre cette expérience de formation agréable et stimulante.

Enfin, je remercie le **Centre Hospitalier de Mende** et son service informatique pour m'avoir accueilli et donné accès à un environnement technique riche et stimulant, dans un domaine où la fiabilité des données a un impact direct sur la qualité de soin.

Annexes

Annexe A – Structure complète du projet `oracle_api`

```
1 oracle_api/
2 |— main.py          # Point d'entrée FastAPI
3 |— config.py        # Configuration et constantes
4 |— database.py       # Pool de connexions Oracle
5 |— schemas.py        # Chargement et validation des schémas JSON
6 |— serializers.py    # Sérialisation des types Oracle
7 |— metrics.py        # Métriques Prometheus
8 |— routes/
9 |   |— data.py       # Endpoint de données (GET /api/tables/{name})
10 |   |— tables.py     # Endpoints de métadonnées (GET /api/tables)
11 |   |— health.py     # Endpoint de santé (GET /health)
12 |— schemas/         # Fichiers JSON de configuration (1 par table, ~87 fichiers)
13 |— docs/
14 |   |— API_DOCUMENTATION.md
15 |— tests/
```

Annexe B – Structure complète du projet `pg_azure_pipeline`

```
1 pg_azure_pipeline/
2 |— main.py          # Point d'entrée du pipeline ETL
3 |— run_api.py        # Serveur API de synchronisation
4 |— pyproject.toml    # Dépendances et métadonnées du projet
5 |— config/
6 |   |— source.json   # Configuration des tables sources
7 |   |— source.schema.json # Schéma de validation
8 |   |— target.json   # Configuration des tables cibles
9 |— src/
10 |   |— config.py      # Configuration applicative
11 |   |— secrets.py     # SecretsManager (Azure Key Vault)
12 |   |— pipeline.py    # Orchestrateur ETL (IngestionPipeline)
13 |   |— extract.py     # APIExtractor + QueryBuilder
14 |   |— transform.py   # DataTransformer
15 |   |— loaders/
16 |       |— base.py    # LoaderStrategy (ABC), LoadResult, IntegrityResult
17 |       |— factory.py # LoaderFactory
18 |       |— simple_upsert.py # PolarsLoader (INSERT ON CONFLICT)
19 |       |— separate_insert_update.py # SeparateInsertUpdateLoader
20 |       |— copy_upsert.py # CopyLoader (COPY + UPSERT) – stratégie retenue
21 |       |— delete_insert.py # DeleteInsertLoader
22 |       |— benchmark.py # Outil de benchmark des stratégies
23 |— tests/
24 |— docs/
```

Annexe C – Structure du projet `sync_oracle_postgresql_clean`

```
1 sync_oracle_postgresql_clean/
2 |— main.py          # Point d'entrée principal
3 |— run_api.py        # Serveur API de synchronisation (port 8001)
4 |— api/
5 |   |— main.py       # Application FastAPI
6 |   |— schemas.py     # Modèles Pydantic pour l'API
7 |   |— sync_service.py # Service de synchronisation
8 |— core/
9 |   |— config_tables.py # Configuration des 84 tables (Pydantic v2)
10 |   |— database_sync.py # Moteur de synchronisation avec chunking adaptatif
11 |   |— connection_manager.py # Gestion des connexions REST API / PostgreSQL
12 |   |— rest_api_connector.py # Client REST API pour la source
13 |— config/           # Fichiers de configuration
14 |— scripts/          # Scripts d'utilisation (interactive_sync.py)
15 |— utils/            # Utilitaires (helpers, performance timer)
16 |— data/             # Données persistantes (état du pipeline)
17 |— logs/            # Fichiers de log (rotation quotidienne)
18 |— CHANGELOG.md      # Historique détaillé des modifications
19 |— tests/
```

Annexe D – Endpoints de l’Oracle Data API

Endpoint	Méthode	Description	Rate Limit
/api/	GET	Informations sur l’API et découverte	100/min
/api/tables	GET	Liste de toutes les tables configurées	60/min
/api/tables/{name}/config	GET	Configuration complète d’une table	60/min
/api/tables/{name}/columns	GET	Définitions des colonnes	60/min
/api/tables/{name}/count	GET	Nombre de lignes	60/min
/api/tables/{name}	GET	Données avec pagination et filtrage	60/min
/health	GET	État de santé de l’API et de la connexion Oracle	–
/metrics	GET	Métriques Prometheus	–

Annexe E – Glossaire

Terme	Définition
BLOB	Binary Large Object – sous-type de LOB pour les données binaires
CLOB	Character Large Object – sous-type de LOB pour les textes longs
COPY	Protocole natif PostgreSQL pour le chargement en masse
DAG	Directed Acyclic Graph – un type de graphique dont les nœuds présentent des liens unidirectionnels qui ne forment pas de cycle.
dlt	Data Load Tool – bibliothèque Python d’ingestion de données
ELT	Extract, Load, Transform – variante où la transformation se fait après le chargement, généralement directement dans la base de données
ETL	Extract, Transform, Load – processus d’intégration de données

HM	Hopital Manager (logiciel métier)
Keyset pagination	Pagination basée sur un curseur (clé + date) plutôt que OFFSET/LIMIT
LOB	Large Object – type Oracle pour les données binaires volumineuses
OLAP	Online Analytical Processing – base optimisée pour les requêtes analytiques
OLTP	Online Transaction Processing – base optimisée pour les transactions
PGI	Progiciel de Gestion Intégré
PoC	Proof of Concept – preuve de concept en français, est une méthode qui permet d'évaluer la faisabilité d'un projet.
RPU	Résumé de Passage aux Urgences
Singer	Standard open-source de connecteurs de données (taps et targets)
UPSERT	Opération combinant INSERT et UPDATE (INSERT ... ON CONFLICT DO UPDATE)

Annexe F – Diagramme de séquence du pipeline d'ingestion

